PBB TEAM GUIDE

# 우리 팀의 첫 GitHub 매뉴얼

파일을 찾고, 코드를 올리고,
검토한 변경을 main에 합치는 방법

## GitHub를 처음 쓰는 팀원을 위해

명령어를 외우기 전에, 지금 보고 있는 브랜치와 파일 위치부터 확인합니다. PBB의 실제 구조를 예로 들어 웹에서 따라 하는 절차를 설명합니다.

**찾기** 내 담당 폴더 → **올리기** 작업 브랜치 → **확인하기** PR · CI → **합치기** main

> **이 매뉴얼이 답하는 질문**
>
> “어디에 무엇을 올리나요?” · “제어 코드는 어디 있나요?”
> “테스트했던 오픈소스는요?” · “CI가 뭔가요?” · “PR을 어떻게 받나요?”

확인 기준: 2026년 11월 9일
저장소: leey00jay/PBB · main 기준 코드 d080d9c
현재 PR #1: 열림, 미병합, 충돌 있음

PBB 저장소 열기

20쪽 구성 · 필요한 장부터 읽어도 됩니다 · 목차와 링크는 클릭할 수 있습니다.

# GUIDE 어디부터 읽으면 될까요?

처음에는 3-4쪽을 읽고, 지금 하려는 작업의 페이지로 이동하세요.


| 지금 궁금한 것 | 쪽 | 설명 |
|---|---|---|
| GitHub가 처음이에요 | 3 | 용어와 화면 읽기 |
| 브랜치와 폴더가 헷갈려요 | 4 | 팀 작업 흐름 |
| 파일이 너무 많아요 | 5-6 | main 폴더 지도와 관리 파일 |
| 제어 코드와 오픈소스가 궁금해요 | 7 | 공식 제어기의 실제 위치 |
| 주행 예제 파일을 알고 싶어요 | 8 | 실행 코드와 결과 기록 |
| 무엇을 올리나요? | 9 | 코드·테스트·설명과 제외 파일 |
| 웹에서 파일을 올리고 싶어요 | 10 | 브라우저 업로드 |
| 내 코드를 main에 제안해요 | 11 | PR 만들기 |
| 지금 PR을 직접 받고 싶어요 | 12-13 | PR #1 병합과 충돌 해결 |
| CI가 뭔가요? | 14 | 자동 검사 읽기 |
| 팀원끼리 어떻게 나누나요? | 15 | 역할과 공통 규격 |
| 내 컴퓨터에서도 작업해요 | 16-17 | 선택: Git 명령어 |
| 버튼이나 파일이 안 보여요 | 18 | 자주 막히는 상황 |
| 매번 확인할 순서가 필요해요 | 19 | 한 장 작업 체크리스트 |
| 근거를 직접 확인해요 | 20 | 저장소·공식 문서 링크 |


### 읽는 기준

**현재 있음**은 실제 확인한 파일, **예시**는 앞으로 만들 때 쓸 이름입니다. 저장소 경로는 PBB 첫 화면을 기준으로 적었습니다. 브랜치 상태와 버튼 모양은 이후 바뀔 수 있습니다.

# 01 GitHub는 공동 작업 공간입니다

파일뿐 아니라 누가, 언제, 왜 바꿨는지도 함께 남깁니다.

**Git**은 내 컴퓨터에서 파일의 변경 기록을 관리하는 도구이고, **GitHub**는 그 기록을 팀과 공유하는 웹 서비스입니다. 웹에서만 간단히 편집할 수도 있고, 내 컴퓨터에서 개발한 뒤 올릴 수도 있습니다.

| 용어 | 쉬운 뜻 | PBB에서의 예 |
|---|---|---|
| 저장소 · Repository | 프로젝트 파일과 변경 기록 전체 | leey00jay/PBB |
| main | 팀이 합친 공통 기준 버전 | 검토한 기능이 모이는 곳 |
| 브랜치 · Branch | 별도로 작업하는 변경 이력 | codex/planning/traffic-light |
| 커밋 · Commit | 설명과 함께 남기는 기록 한 묶음 | “빨간불 정지 판단 추가” |
| Push | 내 컴퓨터의 커밋을 GitHub로 올리기 | 팀원이 변경을 볼 수 있음 |
| Pull | 원격 변경을 받아 현재 브랜치에 반영 | 병합된 main을 내 PC로 받기 |
| PR · Pull Request | 내 변경을 다른 브랜치에 합치자는 요청 | 작업 브랜치에서 main으로 |
| Merge · 병합 | 두 작업 이력을 합쳐 반영 | PR이 main에 들어감 |
| CI | 반복해서 실행하는 자동 검사 | 테스트와 문법 검사 |
| Clone | 파일과 Git 기록을 내 PC에 복제 | 처음 작업 폴더를 만들 때 |

## GitHub 화면에서 자주 쓰는 탭

**Code**: 파일과 브랜치 선택 · **Pull requests**: 합치자는 요청
**Actions**: 자동 검사 결과 · **Issues**: 할 일과 문제 기록
**Settings**: 권한과 보호 규칙 설정. 권한이 있어야 보이거나 바꿀 수 있습니다.

> **저장했다고 전부 끝난 것은 아닙니다**
>
> 내 PC의 파일 저장 → 커밋 → Push는 서로 다릅니다. 웹 편집은 Commit/Propose changes로 GitHub에 기록됩니다. 작업 브랜치에 기록해도 main에는 PR 병합 후 반영됩니다.

근거와 링크: 20쪽

# 02 폴더는 역할, 브랜치는 작업 버전

인지·판단·제어는 폴더로 나누고, 작업 하나마다 브랜치를 만듭니다.

**main** 공통 기준 → **작업 브랜치** 수정 · 검사 → **PR** 리뷰 · CI → **main** 확인 후 병합

> **main 아래에 브랜치 폴더를 만드는 게 아닙니다**
>
> 브랜치 하나에도 프로젝트 전체 파일이 있습니다. 인지 담당 브랜치에서도 제어 폴더를 볼 수 있습니다. 본인 담당 폴더를 중심으로 수정할 뿐입니다.

| 누가 무엇을 바꾸나요? | 작업 브랜치 이름 예시 | 파일 위치 |
|---|---|---|
| 인지: 차선 찾기 | codex/perception/lane | src/pbb/perception/ |
| 판단: 신호에 따른 정지 | codex/planning/traffic-light | src/pbb/planning/ |
| 제어: 속도 추종 조정 | codex/control/speed | src/pbb/control/ |

## 한 번의 작업은 이렇게 끝냅니다

### 01 최신 main에서 시작

한 기능을 정하고 새 브랜치를 만듭니다. 예: 빨간불이면 목표 속도 0으로 결정하기.

### 02 코드·테스트·설명을 함께 수정

같은 기능의 세 파일은 같은 브랜치에서 작업합니다. 파일 하나마다 브랜치를 만들지 않습니다.

### 03 PR에서 확인하고 합치기

base가 main인지 확인하고 CI와 동료 리뷰를 확인합니다. 팀의 기본 병합 방식은 Squash and merge입니다.

### 04 다음 작업은 다시 최신 main에서

병합된 기능을 받은 새 main을 기준으로 새 브랜치를 만듭니다. 이전 작업 브랜치에 계속 새 기능을 쌓지 않습니다.

> **오래 분리해 놓을수록 합치기 어려워집니다**
>
> 인지·판단·제어 브랜치를 각자 오래 쌓기보다, 작게 완성한 기능을 자주 합칩니다. 이미 있는 ye·yeon·yoon 브랜치의 작업은 필요한 변경을 확인해 옮깁니다.

근거와 링크: 20쪽

# 03 main의 폴더 지도

먼저 src → pbb → 내 담당 폴더로 들어가세요.

| 위치 | 무엇을 넣나요? | 현재 상태 |
|---|---|---|
| src/pbb/perception/ | 센서·카메라에서 차선, 장애물 등의 정보를 추출하는 코드 | README와 준비 파일. 자체 실행 코드 이관 전 |
| src/pbb/planning/ | 인지 결과로 정지·감속·경로·목표 속도를 결정하는 코드 | README와 준비 파일. 자체 판단 코드 이관 전 |
| src/pbb/control/ | 목표를 조향·가속·제동 값으로 바꾸는 코드 | controller.py 있음. 공식 제어기 연결·설정 |
| src/pbb/interfaces/ | 영역 간 입력·출력 데이터 형식 | 아직 구체적인 공통 데이터 정의 전 |
| src/pbb/runtime/ | 인지 → 판단 → 제어의 실행 순서를 연결 | 전체 실행 연결은 준비 중 |
| src/pbb/adapters/ | CARLA 센서 읽기, 데이터 변환, 차량 명령 적용 | 프로젝트용 연결 코드 준비 중 |
| tests/ | 동작을 자동으로 확인하는 코드 | 구조 검사와 공식 제어기 테스트 있음 |
| third_party/carla_agents/ | 가져온 공식 오픈소스와 출처 | CARLA 0.9.16 원본 보관 |
| docs/architecture.md | 코드 구조와 영역 연결 규칙 | 설명 문서 있음 |
| Docs/Project Plan/ | 프로젝트 계획과 역할 분담 | main의 README는 빈 계획서 자리 |

> **폴더가 있다는 것과 기능이 완성됐다는 것은 다릅니다**
>
> 현재 main에는 협업 구조와 공식 PID 진입점이 있습니다. 인지·판단·제어가 새 구조에서 전부 연결되어 주행하는 단계는 아직 아닙니다.

**별도 브랜치에 있는 파일**

ye의 decision/decision.py: 기존 판단 코드. 향후 목적지는 planning입니다.
PR #1의 examples/carla_boundary/: 검증한 주행 형태를 보존한 예제입니다.

Docs와 docs는 대소문자가 다른 별도 폴더입니다. 경로를 입력할 때 그대로 쓰세요.

근거: main의 README · 구조 문서 · PR #1. 링크는 20쪽

# 04 관리 파일은 왜 이렇게 많을까요?

대부분은 매번 고치는 파일이 아닙니다. 역할을 알면 충분합니다.

| 파일 | 역할 / 내가 할 일 |
|---|---|
| README.md | 폴더의 첫 안내문. 루트 README부터 읽고, 내 기능의 사용법이 바뀌면 담당 폴더 README를 고칩니다. |
| __init__.py | Python 패키지 구성용 파일. 현재 대부분 준비용이며 실제 기능 코드가 들어 있다는 뜻은 아닙니다. 유지합니다. |
| CONTRIBUTING.md | 작업 시작, 업로드, 검사, PR과 병합에 관한 팀 절차입니다. |
| requirements-control.txt | 공식 제어기 의존성 목록. 현재 carla==0.9.16, numpy==1.26.4를 적어 둡니다. 설치 폴더 대신 이 목록을 공유합니다. |
| tests/test_architecture.py | 인지·판단·제어가 서로의 내부 구현을 직접 가져오는 등 구조 규칙 위반을 검사합니다. |
| tests/control/test_controller.py | 공식 PID 동작과 원본 유지 등을 확인하는 테스트입니다. CARLA 서버 주행 검사는 아닙니다. |
| .github/workflows/ci.yml | GitHub에서 언제 어떤 자동 검사를 돌릴지 적은 설정 파일입니다. |
| .github/pull_request_template.md | PR을 만들 때 나오는 설명 양식입니다. 빈칸을 실제 변경과 검증 결과로 채웁니다. |
| .gitignore | 로컬 Git에서 새로 추적하지 않을 파일 규칙입니다. 이미 추적한 파일이나 웹 업로드를 막아 주는 장벽은 아닙니다. |
| .gitattributes / .editorconfig | 줄바꿈·파일 처리, 들여쓰기·문자 형식 규칙입니다. 일반 기능 작업에서는 보통 그대로 둡니다. |
| LICENSE / UPSTREAM.json | third_party 안의 사용 조건과 원본 출처·버전·해시입니다. 외부 코드를 가져온 근거이므로 보존합니다. |

**확장자 읽기** .py = Python 코드 · .md = 설명 문서 · .yml = 설정
.sh = 셸 실행 스크립트 · .json = 구조화한 데이터 · .png/.jpg = 이미지
.jsonl.gz = 줄 단위 데이터 기록을 압축한 파일

근거와 링크: 20쪽

# 05 테스트했던 오픈소스가 기본입니다

“control에 제어 코드가 없던데?”에 대한 현재 기준 답입니다.

처음에는 폴더 구조만 있어서 제어 코드가 비어 있었습니다. 현재 main에는 **공식 CARLA PID 제어기를 불러오는 파일**을 추가했습니다. 제어 계산 원본은 third_party에 그대로 있습니다.

### 우리 쪽 진입점과 공식 구현은 위치가 다릅니다

**src/pbb/control/controller.py**
공식 클래스를 불러오고 create_controller(vehicle)로 검증한 설정을 적용합니다.

**third_party/carla_agents/agents/navigation/controller.py**
VehiclePIDController의 실제 조향·가속·제동 계산 구현입니다.

**BehaviorAgent**
상황에 따른 주행
→
**LocalPlanner**
경로점 따라가기
→
**PID 제어기**
조향 · 가속 · 제동

| 공식 파일 (agents/navigation 안) | 역할 |
|---|---|
| behavior_agent.py / behavior_types.py | 주행 행동과 행동별 설정 |
| global_route_planner.py | 출발지부터 목적지까지 경로 계산 |
| local_planner.py | 가까운 경로점을 따라가며 제어기를 호출 |
| controller.py | VehiclePIDController와 횡·종방향 PID |
| basic_agent.py / constant_velocity_agent.py | 다른 주행 에이전트 구현. 모두 직접 수정할 필요 없음 |

## 연결할 때 꼭 알아야 하는 두 가지

**속도 단위:** 공식 run_step은 목표 속도를 km/h로 받습니다. 10m/s는 36km/h입니다. 단위를 명시하지 않고 숫자만 넘기면 잘못된 속도 명령이 됩니다.

**제어기 호출:** BehaviorAgent는 내부에서 이미 제어기를 사용합니다. 그 실행에 별도 PID를 하나 더 만들어 차량에 중복 명령을 보내지 않습니다.

기존 예제의 boundary_controller.py는 영상 기반 보조 조향과 이전 시험용 함수입니다. 기본 주행은 공식 BehaviorAgent이며, 영상 보조를 연결한 구성이 검증 대상이었습니다.

근거: 공식 제어기 진입점 · CARLA Agents 문서. 링크는 20쪽

# 06 주행 예제와 결과 파일 찾아보기

현재 examples/carla_boundary/는 PR #1의 작업 브랜치에 있습니다.

| 예제 폴더 안의 파일 | 하는 일 |
|---|---|
| README.md / USAGE.ko.md | 설치 조건, 실행, 조작, 결과 확인 설명. 실행 전에 먼저 읽습니다. |
| run_open_source_autonomy.sh | 공식 에이전트 기반 자동주행을 시작하는 스크립트 |
| open_source_autonomy.py | 공식 BehaviorAgent와 영상 조향을 연결하는 실행 코드 |
| road_boundary_recovery/<br>road_boundary_recovery.py | 카메라 영상에서 도로 경계를 찾는 사용자 제공 검출 코드 |
| boundary_controller.py | 영상 목표점과 보조 조향, 이전 시험용 제어 함수 |
| autonomous_boundary_drive.py<br>run_boundary_carla.py | 각각 정지·출발 반복 시험 / 경계 검출 결과 확인 화면 |
| test_boundary_controller.py | 보조 제어 함수의 자동 테스트 |
| analyze_open_source_run.py<br>analyze_autonomous_test.py | 주행 기록 또는 반복 시험 결과를 분석해 보고서 생성 |
| launch_carla.py / stop_carla.py<br>configure_carla.py | 서버 시작·종료와 시작 설정. 환경 조건은 README 참조 |
| setup_carla_swap.sh | Linux 스왑 설정용 선택 도구. 일반 주행 때 실행할 파일이 아닙니다. |
| requirements-autonomy.txt<br>third_party/carla_agents/ | 추가 의존성 목록 / 검증 당시 공식 소스 보존본 |

## validation/날짜-시간/은 실행 결과 보관함입니다

**검증결과.txt**: 사람이 읽는 보고서 · **metrics.png**: 그래프
**summary.json**: 요약 수치 · **metadata.json**: 실행 조건
**verification.json**: 당시 기록 검사 결과
**frames.jsonl.gz**: 압축한 상세 기록 · **.jpg**: 대표 장면

### 기록을 읽는 순서

먼저 보고서와 그래프를 보세요. 2026-11-09 기록은 180초, 정지·출발 각 4회, 회전 3회, 충돌 0회였습니다. 낮·저속·다른 차량과 보행자 없는 조건이며, 카메라만으로 모든 것을 인식한 시험은 아닙니다.

예제·검증 보고서 링크: 20쪽

# 07 올릴 것은 코드 + 테스트 + 설명

판단 담당자가 빨간불 정지 기능을 추가하는 예시입니다.

| 함께 올릴 파일 예시 | 내용 |
|---|---|
| src/pbb/planning/decision.py | 어떤 입력에서 정지·감속·주행을 결정하는지 구현 |
| tests/planning/test_decision.py | 빨간불 입력이면 목표 속도 0이 되는지 등을 확인 |
| src/pbb/planning/README.md | 입력, 출력, 단위, 실행 방법, 아직 안 되는 조건 설명 |

위 decision.py와 test_decision.py는 **새 기능을 만들 때의 예시 이름**입니다. main의 해당 폴더에 이미 완성되어 있다는 뜻이 아닙니다. README는 기존 파일을 수정합니다.

| 올려도 되는 것 | 보통 올리지 않는 것 |
|---|---|
| 직접 수정한 .py, 테스트, 문서 | CARLA 설치 폴더 전체, .venv/, .carla-python/, .autonomy-deps/ |
| 의존성 이름·버전 목록 | 설치된 패키지 폴더 전체 |
| 작은 예시 입력과 선택한 결과 보고서 | 원본 대용량 영상, 전체 데이터셋, 매 실행마다 생성되는 로그 |
| 비밀값 없는 설정 예제 | 비밀번호, 인증 토큰, 개인 키, 비밀값이 든 .env |

## 올리기 전에 세 가지만 확인하세요

### 01 파일 위치

planning 폴더에 판단 코드를 올립니다. PBB 전체를 ZIP으로 올리거나 PBB/PBB처럼 중첩시키지 않습니다.

### 02 원본과 결과 구분

프로그램을 실행해서 생긴 영상과 설치 폴더는 코드가 아닙니다. 큰 자료는 보관 위치와 재현 방법을 문서에 남깁니다.

### 03 필요 없는 파일과 비밀값

업로드 목록을 직접 읽습니다. .gitignore는 불필요한 파일을 모두 알아서 판별하거나 웹 업로드를 차단하지 않습니다.

**파일 크기:** 브라우저 업로드는 파일당 25MiB까지입니다. 큰 자료는 별도로 보관하고 링크를 공유합니다. 원본 코드는 파일 그대로 올려야 비교하기 쉽습니다.

근거: GitHub 파일 추가 안내. 링크는 20쪽

# 08 웹에서 파일 올리기

터미널을 몰라도 시작할 수 있습니다. 쓰기 권한이 있는 계정으로 로그인하세요.

### 01 PBB에서 main 선택

Code 탭의 파일 목록 위 브랜치 메뉴를 엽니다. 처음 작업을 시작할 때는 main을 선택합니다.

### 02 파일을 둘 폴더로 이동

판단 코드라면 src → pbb → planning 순서로 들어갑니다. 업로드 위치가 파일의 저장 위치가 됩니다.

### 03 Add file → Upload files

내 컴퓨터의 decision.py를 선택하거나 끌어 놓습니다. 이 파일 하나를 올릴 때 상위 PBB 폴더를 통째로 끌어 놓지 않습니다.

### 04 변경 설명과 새 브랜치 선택

예: “빨간불 정지 판단 추가”. main에 직접 기록하는 대신 새 브랜치를 만드는 옵션을 선택하고 codex/planning/traffic-light처럼 이름을 적습니다.

### 05 Propose changes로 기록

선택한 옵션에 따라 Commit changes로 보일 수도 있습니다. 저장 후 브랜치 메뉴에 작업 브랜치 이름이 보이는지 확인합니다.

### 06 같은 작업 브랜치에서 나머지도 추가

tests/planning에 테스트를 올리고 planning의 README를 수정합니다. 이번에는 현재 작업 브랜치에 커밋합니다. 같은 기능에 브랜치를 계속 새로 만들지 않습니다.

### 07 Files 목록에서 위치와 내용 확인

src/pbb/planning/decision.py, 테스트, README가 모두 같은 브랜치에 있는지 확인합니다. 그다음 11쪽의 PR을 만듭니다.

> **이미 있는 파일을 수정하려면**
>
> 작업 브랜치를 선택한 뒤 파일을 열고 연필 모양의 Edit 버튼을 누릅니다. 변경을 저장할 때도 현재 작업 브랜치인지 확인하세요.

> **내 컴퓨터 폴더에 복사만 하면 올라가나요?**
>
> 아니요. 웹으로 업로드하거나, 로컬 Git에서 커밋과 Push를 해야 합니다. 어느 PC에서든 폴더가 자동 동기화된다고 가정하지 마세요.

버튼 이름 확인: GitHub 파일 추가 안내. 링크는 20쪽

# 09 PR은 “이 변경을 합쳐 주세요”입니다

작업 브랜치에 올라간 파일을 main으로 제안합니다.

### 01 Pull requests → New pull request

업로드 후 Compare & pull request 배너가 보이면 그 버튼으로 시작해도 됩니다.

### 02 base와 compare 확인

**base: main**은 변경을 받을 곳입니다.
**compare: 내 작업 브랜치**는 변경을 가져올 곳입니다.

```
base: main  <-  compare: codex/planning/traffic-light
```

### 03 Files changed에서 변경 범위 확인

추가된 줄은 보통 초록색, 삭제된 줄은 빨간색입니다. 수정도 기존 줄 삭제 + 새 줄 추가로 보입니다. 의도하지 않은 파일이 섞이지 않았는지 읽습니다.

### 04 제목과 설명을 채우고 Create pull request

제목은 “수정”보다 “빨간불이면 목표 속도 0으로 설정”처럼 적습니다. 아직 작업 중이면 Draft PR을 사용합니다.

## PR 설명 예시 - 실제로 수행한 것만 적습니다

> **제목: 빨간불이면 목표 속도를 0으로 설정**
>
> **목적**: 신호 정지 판단 추가
> **영향**: planning 코드와 테스트, README
> **입출력**: 신호 상태를 입력받아 목표 속도 km/h를 출력
> **검증**: 정지·주행 입력 테스트 결과와 실행 명령 기재
> **제한**: 이 예시는 CARLA 전체 주행을 검증한 것이 아님

PBB에는 PR 양식이 준비되어 있습니다. 변경 목적, 영향 영역, 연결·호환성, 검증을 채우고 확인한 체크박스만 선택합니다. **현재 PR #1의 설명은 양식이 남아 있어, 병합 전에 실제 내용으로 보완하는 것이 좋습니다.**

## 리뷰에서 수정 요청을 받았다면

같은 작업 브랜치에 수정 내용을 추가 커밋하고 Push합니다. 기존 PR이 갱신되므로, 같은 일을 위한 새 PR을 매번 만들지 않습니다. 검사도 새 커밋 기준으로 다시 확인합니다.

근거: GitHub PR 만들기 · GitHub flow. 링크는 20쪽

# 10 지금 PR #1을 main으로 받는 방법

“받기”에는 GitHub에서 병합하기와 내 PC로 내려받기가 있습니다.

실제 PR #1 열기

**현재 상태 · 2026-11-09 확인**

제목: Add CARLA boundary autonomy demo and validation results
받을 곳: **main**
가져올 곳: **codex/integration/carla-autonomy-demo**
상태: **Open · 아직 미병합 · 충돌 있음**

### 01 PR의 내용과 방향 확인

Conversation에서 목적과 검증 결과를 읽고, Files changed에서 추가·수정 파일을 확인합니다. base가 main인지 확인합니다.

### 02 충돌이 있다면 먼저 해결

현재 PR #1은 바로 병합할 수 없습니다. 13쪽 순서대로 충돌을 해결하고 Commit merge로 작업 브랜치를 갱신해야 합니다.

### 03 CI와 동료 리뷰 확인

충돌 해결 후 새 커밋에 대한 검사가 끝날 때까지 기다립니다. 실패가 있으면 원인을 고칩니다. 필요한 리뷰 요청도 반영합니다.

### 04 Squash and merge 선택

PR 아래 병합 버튼의 드롭다운에서 Squash and merge를 선택합니다. 여러 작업 커밋을 main에는 하나의 변경 묶음으로 남기는 방식입니다.

### 05 Confirm squash and merge

제목과 내용을 확인하고 최종 확정합니다. **이 단계에서 변경이 main에 들어갑니다.** 보라색 Merged 표시를 확인합니다.

### 06 main에서 파일 확인

Code 탭 → main을 선택해 examples/carla_boundary가 보이는지 확인합니다. PC의 파일까지 바뀌는 것은 아니므로 로컬은 16쪽대로 갱신합니다.

**이름이 비슷해도 결과가 다릅니다**

**Commit merge**: 충돌 해결 내용을 작업 브랜치에 기록
**Confirm squash and merge**: PR 변경을 main에 반영
**Close pull request**: 요청을 닫기. 코드 병합은 아님

근거: 실제 PR #1 · GitHub PR 병합 안내. 링크는 20쪽

# 11 충돌은 “어느 내용을 남길까요?”입니다

두 브랜치가 같은 부분을 다르게 고치면 Git이 자동으로 결정하지 못합니다.

현재 PR #1에서 충돌한 파일은 **README.md**, **docs/architecture.md**, **.github/workflows/ci.yml**입니다. 파일이 망가졌다는 뜻은 아닙니다. 두 작업의 의도를 읽고 최종 내용을 작성해야 합니다.

```
<<<<<<< current-branch
예제 실행 안내
=======
공식 제어기 위치 안내
>>>>>>> main
```

위는 이해를 위한 예시이며 실제 파일 내용은 아닙니다. 두 안내가 모두 필요하면 모두 남기고, <<<<<<< / ======= / >>>>>>> 표식은 제거합니다. 서로 모순되는 코드는 단순히 둘 다 남기면 안 됩니다.

### 01 PR 아래 Resolve conflicts

활성화되어 있으면 GitHub의 충돌 편집기를 엽니다. 비활성화되어 있으면 로컬 Git이나 도움을 받아 해결해야 합니다.

### 02 파일마다 최종 내용 작성

한쪽 버전을 통째로 고르는 대신 아래 보존 기준을 확인합니다. 각 파일의 충돌 구간을 전부 해결합니다.

| 충돌 파일 | 이번에 함께 보존할 내용 |
|---|---|
| README.md | main의 초보자 안내·공식 제어기 설명 + 예제 위치와 사용 안내 |
| docs/architecture.md | 현재 공식 제어기 구조 + 예제의 향후 모듈 이관 설명 |
| .github/workflows/ci.yml | main의 의존성·경로·공식 코드 검사 + 예제 테스트·문법 검사 |

### 03 Mark as resolved → Commit merge

각 파일을 해결 완료로 표시하고, 모두 끝나면 Commit merge를 누릅니다. 이 단계는 main의 내용을 PR 작업 브랜치에 합칩니다.

### 04 새 검사 확인 후 12쪽의 최종 병합

CI 설정 파일은 들여쓰기까지 확인해야 합니다. 잘 모르겠다면 충돌 파일 목록과 PR 링크를 공유해 도움을 받고, 최종 병합은 검사 후 진행합니다.

근거: PR #1 두 버전 비교 · GitHub 충돌 해결 안내. 링크는 20쪽

# 12 CI는 자동으로 풀어 보는 시험입니다

Continuous Integration, 지속적 통합. 자주 합치고 자동 검사로 오류를 찾습니다.

**코드 업로드** Push 또는 PR → **GitHub Actions** 검사 실행 → **결과 확인** 통과 · 실패

GitHub Actions는 자동 작업을 실행하는 기능이고, **CI는 그 기능으로 코드가 함께 동작하는지 반복 확인하는 방식**입니다. 검사 항목은 .github/workflows/ci.yml에 적혀 있습니다.

| PBB main의 검사 | 확인하는 것 |
|---|---|
| Python 3.10 / 3.12 | 두 Python 버전에서 검사 실행 |
| 의존성 설치 | requirements-control.txt에 적힌 패키지 준비 |
| unittest | 모듈 연결 규칙, 공식 PID 동작·원본 유지 등 |
| compileall | Python 파일의 문법 오류 확인 |

현재 설정은 main 대상 PR과 main·codex/** 브랜치 Push에서 실행됩니다. main 기준 테스트는 최근 확인 시 18개였습니다. 이후 기능이 추가되면 개수와 항목도 바뀝니다.

## 결과는 어디서 보나요?

### 01 PR의 Checks 또는 아래 검사 목록

초록색은 통과, 빨간색은 실패입니다. 진행 중·대기 상태면 결과를 기다립니다. 검사 미실행이나 취소는 통과와 다릅니다.

### 02 실패한 검사 상세 열기

Actions 탭에서도 실행 기록을 볼 수 있습니다. 실패한 작업과 단계의 로그를 읽고 오류 메시지를 확인합니다.

### 03 고친 뒤 같은 브랜치에 다시 올리기

새 커밋에 대한 CI 결과를 확인합니다. 이전 커밋의 초록 표시만 보고 병합하지 않습니다.

> **초록색이어도 전체 자율주행 검증은 별도입니다**
>
> 현재 main의 CI는 CARLA 서버를 켜서 주행하지 않습니다. 알고리즘 변경에는 관련 테스트가 필요하고, 주행 성능은 시뮬레이터 시나리오와 결과 기록으로 별도 확인합니다.

PR 검사가 막혔다면 충돌을 먼저 해결한 뒤 최신 CI를 확인합니다.

근거: main의 ci.yml · GitHub CI 안내. 링크는 20쪽

# 13 나중에 쉽게 합치는 팀 약속

파일을 나누는 것보다 서로 주고받는 값의 약속이 먼저입니다.

| 역할 | 입력 예시 | 출력 예시 |
|---|---|---|
| 인지 · perception | 카메라 프레임, 센서값 | 차선·장애물 정보, 신뢰도 |
| 판단 · planning | 인지 결과, 차량 상태 | 주행 상태, 경로, 목표 속도 |
| 제어 · control | 목표 속도·경로점, 차량 상태 | 조향·가속·브레이크 값 |

이 표는 역할 설명용이며 공통 데이터 형식은 준비 중입니다. 공식 에이전트와 자체 기능의 연결 위치를 정한 뒤 interfaces와 adapters에서 형식을 맞춥니다.

> **데이터 약속 예시 - 팀 합의 후 구현**
>
> target_speed_kmh: km/h 단위 목표 속도
> timestamp_s: 어떤 시계를 기준으로 한 초인지 명시
> lane_confidence: 0~1인지 다른 범위인지 명시
> 검출 실패: None인지 별도 상태인지 명시
> 좌표: 차량 기준인지 지도 기준인지 명시

## 우리 팀 기본 작업 규칙

### 01 하나의 기능은 하나의 짧은 PR

큰 폴더 이동, 전체 코드 정렬, 새 기능을 한 번에 섞지 않습니다. PR 제목만 봐도 변경 목적을 알 수 있게 합니다.

### 02 공통 규격부터 합의하고 먼저 반영

데이터 형식 변경은 영향을 받는 담당자와 먼저 맞춥니다. 다른 영역의 내부 함수를 직접 가져다 쓰지 않고 runtime에서 연결합니다.

### 03 공통 파일 수정은 미리 공유

의존성, CI, README, 전체 실행 파일은 겹치기 쉬우므로 변경 범위를 미리 공유합니다.

### 04 작성자와 확인 역할 구분

작성자는 코드·검증 결과를 준비하고, 동료는 Files changed와 결과를 확인합니다. 병합 담당자가 최신 CI와 미해결 의견을 확인합니다.

**팀 규칙과 보호 설정은 별개입니다.** 문서만으로 직접 Push가 막히지 않습니다. main 보호와 필수 검사·리뷰는 Settings에서 따로 설정해야 하며, 현재 적용 완료로 확인된 상태가 아닙니다.

근거: CONTRIBUTING.md · 구조 문서. 링크는 20쪽

# 14 선택: 내 컴퓨터로 받기

웹만 쓰는 팀원은 이 장을 건너뛰어도 됩니다. 아래 명령은 Linux·Bash 기준입니다.

내 컴퓨터에서 Python 코드를 수정·실행하려면 Git으로 복제한 작업 폴더가 편합니다. **Git 설치와 GitHub 인증**이 먼저 필요합니다. 명령은 한 줄씩 읽고 실행하세요.

## A. 처음 작업 폴더를 만들 때 한 번만

새 폴더를 만들 상위 위치에서 실행합니다. 아래 이름의 폴더가 이미 있으면 다른 이름을 쓰세요.

```
git clone https://github.com/leey00jay/PBB.git PBB-work
cd PBB-work
git status
```

clone은 GitHub 파일과 변경 기록을 복제합니다. **Download ZIP**은 선택한 버전의 파일만 받으므로, 이후 커밋·브랜치·Push 작업용 clone과 다릅니다.

## B. PR이 병합된 최신 main을 받을 때

이미 clone한 작업 폴더에서 실행합니다. 먼저 git status를 보고 작업 중인 변경을 자신의 작업 브랜치에 커밋하거나 안전하게 보관한 뒤 진행합니다.

```
git status
git switch main
git pull --ff-only origin main
```

**origin**은 GitHub 저장소 연결의 기본 이름입니다. **--ff-only**는 이력이 갈라져 있으면 임의로 합치지 않고 멈추게 합니다. 실패하면 오류를 읽고 도움을 요청하세요.

## C. 새 작업 시작

```
git switch -c codex/planning/traffic-light
```

이제 새 작업 브랜치에서 파일을 수정합니다. 브랜치 이름이 이미 있다면 새로 만들지 말고 같은 작업인지 확인하세요.

> **기존 폴더가 여러 개라면**
>
> 지금 어느 폴더를 편집하는지 먼저 확인합니다. git remote -v로 PBB 연결을, git status로 브랜치와 변경 파일을 확인합니다. 기존 파일이 있는 폴더에 새 clone을 덮어씌우거나 무조건 초기화하지 않습니다.

PBB의 명령어 절차: CONTRIBUTING.md, 링크는 20쪽

# 15 선택: 검사하고 커밋·Push하기

PBB 작업 폴더에서 실행합니다. 코드 검사는 Python 3.10 또는 3.12 기준입니다.

## 1. 가상환경과 의존성 준비

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-control.txt
```

가상환경 생성·설치는 새 환경에서 준비할 때 합니다. 다음에 같은 환경을 쓰면 activate부터 실행합니다. Python 버전은 python3 --version으로 확인합니다.

## 2. 현재 main 구조의 기본 검사

```
PYTHONPATH=src:third_party/carla_agents \
  python -m unittest discover -s tests -v
python -m compileall -q src tests third_party/carla_agents
git diff --check
```

첫 명령은 두 줄로 표시했으며 줄 끝의 역슬래시까지 함께 입력합니다. 주행 예제를 바꿨다면 해당 README의 추가 검사도 실행합니다.

## 3. 변경 파일을 확인하고 필요한 것만 기록

```
git status
git diff
git add src/pbb/planning/decision.py
git add tests/planning/test_decision.py
git add src/pbb/planning/README.md
git diff --cached
git commit -m "Add traffic light stop decision"
git push -u origin HEAD
```

파일명은 이번 판단 기능 예시입니다. **실제로 수정한 경로**로 바꾸세요. add는 커밋할 파일 선택, commit은 로컬 기록, push는 GitHub 업로드입니다. 그다음 11쪽대로 PR을 만듭니다.

> **최신 main을 작업 중인 브랜치에 반영하려면**
>
> 변경을 먼저 커밋하고 현재 작업 브랜치에서 git fetch origin → git merge origin/main 순서로 진행합니다. 충돌을 해결했다면 해당 파일을 add·commit하고 검사를 다시 실행한 뒤 Push합니다. 공유 브랜치에 force push는 기본 절차로 쓰지 않습니다.

PBB의 검사·동기화 절차: CONTRIBUTING.md, 링크는 20쪽

# 16 자주 막히는 상황

화면의 브랜치 이름과 파일 경로부터 확인하면 대부분 원인을 좁힐 수 있습니다.

| 상황 | 먼저 확인할 것 / 다음 행동 |
|---|---|
| “올렸는데 main에 없어요.” | 작업 브랜치에만 커밋했을 수 있습니다. PR을 만들었는지, Merged인지 확인합니다. |
| “다른 사람 화면에는 파일이 있어요.” | 브랜치가 같은지 확인합니다. 예제는 현재 PR #1 브랜치에, 기존 판단 코드는 ye에 있습니다. |
| “control 폴더 파일이 짧아요.” | 공식 제어기를 불러오는 진입점입니다. 실제 계산 구현은 third_party/.../navigation/controller.py에 있습니다. |
| GitHub는 바뀌었는데 PC는 그대로예요. | 저장소 폴더·브랜치를 확인한 후 최신 main을 Pull합니다. 웹 병합은 내 PC 파일을 자동 갱신하지 않습니다. |
| “Merge 버튼을 못 눌러요.” | 충돌, CI 실패·대기, Draft, 필요한 리뷰, 권한 문제를 PR 아래 안내문에서 확인합니다. |
| “Upload/Edit 버튼이 없어요.” | 로그인 계정과 저장소 쓰기 권한을 확인합니다. 팀원이면 소유자가 협업자로 초대했는지 확인합니다. |
| “커밋은 했는데 팀원이 못 봐요.” | 로컬 커밋 후 Push까지 했는지, Push한 원격 저장소와 브랜치가 맞는지 확인합니다. |
| “파일이 Git 목록에 안 나와요.” | 저장 여부와 .gitignore 규칙을 확인합니다. 이미 추적된 비밀 파일은 ignore 추가만으로 지워지지 않습니다. |
| “Push가 rejected라고 나와요.” | 서버에 새 변경이 있을 수 있습니다. 내 변경을 보존하고 원격 이력을 확인해 합칩니다. force로 덮어쓰지 않습니다. |
| “내가 잘못 합쳤어요.” | 어떤 PR에서 문제가 생겼는지 기록합니다. main 이력을 지우는 대신 되돌리는 변경(Revert)을 검토하고 새 PR로 확인합니다. |

**도움을 요청할 때 이 네 가지를 보내세요**

1) 저장소·PR 링크 2) 현재 브랜치 이름
3) 하려던 일과 수정한 파일 경로 4) 오류 문구 또는 실패한 CI 단계
비밀번호·토큰은 보내지 않습니다.

근거와 링크: 20쪽

# QUICK SHEET 매번 이 순서만 확인하세요

이 페이지는 팀원이 작업을 시작할 때 옆에 두고 보는 요약입니다.

## 작업 시작 전

01 최신 main을 기준으로 시작했나요?
02 이번에 할 기능 하나와 담당 폴더를 정했나요?
03 다른 담당자에게 필요한 입력·출력과 단위를 맞췄나요?
04 같은 작업용 브랜치에서 수정하고 있나요?

## PR 만들기 전

05 코드·테스트·설명이 함께 있나요?
06 설치 폴더, 큰 영상, 비밀값이 업로드에 섞이지 않았나요?
07 실제로 실행한 검사와 결과를 설명에 적었나요?
08 base: main, compare: 작업 브랜치가 맞나요?

## main에 합치기 전

09 Files changed를 읽고 필요한 변경만 있는지 확인했나요?
10 충돌을 해결하고 최신 커밋의 CI를 확인했나요?
11 동료 리뷰와 남아 있는 요청을 확인했나요?
12 Squash and merge 후 Merged 표시를 확인했나요?

## 합친 뒤

13 main에서 새 파일과 내용을 확인했나요?
14 로컬 작업자는 최신 main을 받았나요?
15 끝난 작업 브랜치에 새 일을 이어 쌓지 않나요?

**코드 위치**
src/pbb/담당

→

**검사 위치**
tests/담당

→

**설명 위치**
담당 README

> **PR #1에 바로 적용한다면**
>
> 설명 보완 → Files changed 확인 → 충돌 3개 파일 해결 → 최신 CI·리뷰 확인 → Squash and merge → main 확인 순서입니다. 이 문서는 절차를 안내하며, 작성 과정에서 PR을 병합하지 않았습니다.

병합 후 Delete branch는 완료한 작업 브랜치를 정리하는 선택입니다. 이미 main에 합친 파일이 삭제되는 것은 아닙니다. 다른 미완료 작업이 있는 브랜치는 먼저 확인하세요.

# REFERENCE 근거와 바로가기

문서의 저장소 상태는 2026-11-09 확인 기준이며, 실시간 상태는 링크에서 확인하세요.

## PBB 프로젝트 - 실제 위치와 상태

PBB 첫 화면 / 현재 main

이 매뉴얼의 main 기준 스냅샷: d080d9c

README: 초보자 안내와 폴더 지도

CONTRIBUTING: 팀 작업·검사·병합 절차

architecture: 영역 분리와 이관 순서

control/controller.py: 공식 제어기 진입점

ci.yml: main 기준 자동 검사 설정

PR #1: 주행 예제와 검증 결과 / 현재 상태 확인

예제 보존본: d11968a / examples/carla_boundary

20261109-214402 검증 보고서

ye 브랜치의 기존 판단 코드

## 공식 안내 - 화면 절차와 개념

GitHub flow: 브랜치에서 작업하고 PR로 합치기

GitHub: 파일 추가와 업로드

GitHub: PR 만들기

GitHub: PR 병합하기

GitHub: 웹에서 병합 충돌 해결하기

GitHub: CI 알아보기

CARLA 0.9.16: 공식 Agents 설명

CARLA 0.9.16: 공식 Agents 원본

> **문서를 업데이트할 때**
>
> 폴더 구조, CI, PR 상태가 바뀌면 이 안내도 함께 갱신하세요. 주행 검증 결과는 해당 기록의 조건에서만 해석합니다. CARLA 원본의 MIT 라이선스는 사용자 제공 코드나 PBB 전체에 자동 적용되지 않습니다.